Задание 1: aunt, uncle, ancestor

Дано: `parent(X,Y)` (X — родитель Y), `person(X,G)` (X — человек с полом G).

```
% братья/сестры
sibling(X,Y) :- parent(Z,X), parent(Z,Y), not(X=Y).

% aunt: X — тётя Y, если X женского пола и X — сиблинг
родителя Y
aunt(X,Y) :- person(X,female), sibling(X,Z), parent(Z,Y).

% uncle: X — дядя Y, если X мужского пола и X — сиблинг
родителя Y
uncle(X,Y) :- person(X,male), sibling(X,Z), parent(Z,Y).

% ancestor: рекурсивное замыкание parent
ancestor(X,Y) :- parent(X,Y).
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
```

Задание 2: безопасность и стратифицированность

№1

```
sister(X,Y) :- parent(Z,X), parent(Z,Y), female(X),
not(X=Y).
aunt(X,Y)   :- parent(Z,Y), sister(X,Z).
```

Все переменные в головах и в отрицательных литералах связаны положительными литералами тела.

- `sister`: X ∈ parent(Z,X) ✓, Y ∈ parent(Z,Y) ✓, Z ∈ parent(Z,X) ✓, `not(X=Y)` — X и Y связаны ✓.
- `aunt`: X ∈ sister(X,Z) ✓, Y ∈ parent(Z,Y) ✓, Z ∈ parent(Z,Y) ✓.

Нет рекурсии через отрицание (`not(X=Y)` — встроенное неравенство, не IDB-предикат). Страты: 0 — EDB (parent, female), 1 — sister, 2 — aunt. Стратифицирована.

№2

```
root(X) :- not(parent(Y,X)).
same_generation(X,Y) :- root(X), root(Y).
same_generation(X,Y) :- parent(V,X), parent(W,Y),
same_generation(V,W).
```

В правиле `root(X) :- not(parent(Y,X))` — ни X, ни Y не связаны ни одним положительным литералом тела. Небезопасна.

При этом `root` отрицает только EDB-предикат `parent`, рекурсия в `same_generation` положительная — отрицательных циклов нет. Страты: 0 — EDB (parent), 1 — root, 2 — same_generation. Стратифицирована, но не безопасна.

№3

```
friend(X,Y) :- friend(Y,X).
friend(X,Y) :- friend(X,Z), friend(Z,Y).
enemy(X,Y)  :- enemy(Y,X).
enemy(X,Y)  :- friend(X,Z), enemy(Z,Y), not(friend(X,Y)).
connected_after(X,Y,U,V) :- friend(X,U), friend(Y,V).
connected_after(X,Y,U,V) :- friend(X,V), friend(Y,U).
connects_new_enemies(X,Y) :- connected_after(X,Y,U,V),
enemy(U,V), not(friend(U,V)).
potential_friend(X,Y) :- person(X), person(Y),
                         not(enemy(X,Y)),
not(connects_new_enemies(X,Y)).
```

Во всех правилах каждая переменная связана положительным литералом тела:

- `enemy(X,Y) :- friend(X,Z), enemy(Z,Y), not(friend(X,Y))`: X,Z из friend(X,Z) ✓, Y из enemy(Z,Y) ✓, not(friend(X,Y)) — X,Y связаны ✓.
- `potential_friend`: X,Y из person(X), person(Y) ✓.

Безопасна.

Граф зависимостей:

Откуда	Знак	Куда
friend	+	friend

Откуда	Знак	Куда
friend	+	enemy
friend	–	enemy
enemy	+	enemy
friend	+	connected_after
connected_after	+	connects_new_enemies
enemy	+	connects_new_enemies
friend	–	connects_new_enemies
enemy	–	potential_friend
connects_new_enemies	–	potential_friend

Отрицательных циклов нет (enemy не зависит от connects_new_enemies или potential_friend). Стратифицирована.

Страты:

- Страт 0: EDB (person и т.д.)
- Страт 1: {friend, connected_after}
- Страт 2: {enemy, connects_new_enemies} (enemy отрицает friend из страта 1; connects_new_enemies отрицает friend из страта 1 и использует enemy из страта 2)
- Страт 3: {potential_friend} (отрицает enemy и connects_new_enemies из страта 2)

Задание 3: минимальная модель программы bi/even

```
bi(X,Y)   :- g(X,Y).
bi(Y,X)   :- g(X,Y).
even(X,Y) :- bi(X,Z), bi(Z,Y).
even(X,Y) :- bi(X,U), bi(U,V), even(V,Y).
```

g = {(a,b), (b,c), (c,d)}

bi = g ∪ reverse(g):

```
bi = {(a,b),(b,a),(b,c),(c,b),(c,d),(d,c)}
```

Это граф-цепочка a↔b↔c↔d.

Итерация 1 (правило `even :- bi(X,Z), bi(Z,Y)` — пути длины 2):

bi(X,Z)	bi(Z,Y)	even(X,Y)
(a,b)	(b,a)	(a,a)
(a,b)	(b,c)	(a,c)
(b,a)	(a,b)	(b,b)
(b,c)	(c,b)	(b,b)
(b,c)	(c,d)	(b,d)
(c,b)	(b,a)	(c,a)
(c,b)	(b,c)	(c,c)
(c,d)	(d,c)	(c,c)
(d,c)	(c,b)	(d,b)
(d,c)	(c,d)	(d,d)

```
even₁ = {(a,a),(a,c),(b,b),(b,d),(c,a),(c,c),(d,b),(d,d)}
```

Итерация 2 (правило `even :- bi(X,U), bi(U,V), even(V,Y)`) не добавляет новых пар — неподвижная точка достигнута.

Минимальная модель:

```
bi   = {(a,b),(b,a),(b,c),(c,b),(c,d),(d,c)}
even = {(a,a),(a,c),(b,b),(b,d),(c,a),(c,c),(d,b),(d,d)}
```

even(X,Y) выполняется тогда и только тогда, когда X и Y находятся на чётном расстоянии в цепочке a–b–c–d.

Добавление `onlyodd(X,Y) :- not(even(X,Y))`

В правиле `onlyodd(X,Y) :- not(even(X,Y))` переменные X и Y встречаются только в отрицательном литерале — не связаны никаким положительным литералом. Небезопасна.

Исправление:

```
onlyodd(X,Y) :- bi(X,Y), not(even(X,Y)).
```

X и Y теперь связаны bi(X,Y).

bi \ even = {(a,b),(b,a),(b,c),(c,b),(c,d),(d,c)} — ни один из них не входит в even.

```
onlyodd = {(a,b),(b,a),(b,c),(c,b),(c,d),(d,c)}
```

Минимальная модель единственна. Страты: 1 — {bi, even}, 2 — {onlyodd} (использует not(even)).

Задание 4: программа с u, v, t, r, s

u = {(a),(b),(c)}, v = {(b),(c),(d)}

```
t(X,Y) :- u(X), u(Y), not(v(X)).
r(X)   :- u(X), v(Y), not(t(X,Y)).
s(X)   :- r(Y), t(Y,X), not(r(X)).
```

(a) Безопасность и стратифицированность

- `t(X,Y)`: X ∈ u(X) ✓, Y ∈ u(Y) ✓, not(v(X)) — X связан ✓.
- `r(X)`: X ∈ u(X) ✓, Y ∈ v(Y) ✓, not(t(X,Y)) — X,Y связаны ✓.
- `s(X)`: Y ∈ r(Y) ✓, X ∈ t(Y,X) ✓, not(r(X)) — X связан ✓.

Безопасна.

t использует not(v) — v это EDB, нет цикла; r использует not(t); s использует not(r). Отрицательных циклов нет — стратифицирована.

Страты: 0 — EDB (u, v), 1 — {t}, 2 — {r}, 3 — {s}.

(b) Стратифицированная модель

Страт 1 — вычисляем t:

u \ v = {a} (b,c ∈ v; a ∉ v). Только X=a проходит:

```
t = {(a,a), (a,b), (a,c)}
```

Страт 2 — вычисляем r:

Для каждого X ∈ u проверяем, существует ли Y ∈ v с (X,Y) ∉ t:

- X=a: t(a,b)✓, t(a,c)✓, но t(a,d)? u(d) ложно, значит (a,d) ∉ t → not(t(a,d)) истинно → r(a)
- X=b: b ∈ v → not(v(b)) ложно → t(b,Y) не выводится → not(t(b,b)) истинно → r(b)
- X=c: аналогично → r(c)

```
r = {a, b, c}
```

Страт 3 — вычисляем s:

r = {a,b,c}, t = {(a,a),(a,b),(a,c)}

- Y=a: t(a,a) → X=a, но r(a) ∈ r → not(r(a)) ложно
- Y=a: t(a,b) → X=b, но r(b) ∈ r → not(r(b)) ложно
- Y=a: t(a,c) → X=c, но r(c) ∈ r → not(r(c)) ложно
- Y=b,c: t(b,*) и t(c,*) — пусто

```
s = {}
```

Итоговая модель:

```
t = {(a,a), (a,b), (a,c)}
r = {a, b, c}
s = {}
```

(c) Другая минимальная модель

```
t = {(a,a), (a,b), (a,c), (a,d)}
r = {b, c}
s = {}
```

Почему это модель:

- Правило t: u(a)∧u(Y)∧¬v(a) → t(a,Y) для Y∈{a,b,c} ✓. Факт t(a,d) добавлен «вручную» — правило не требует его вывода (u(d) ложно), но и не запрещает.
- r(a): нужно ∃Y∈v: ¬t(a,Y). Но t(a,b)✓, t(a,c)✓, t(a,d)✓ — все Y∈v={b,c,d} покрыты → r(a) не выводится ✓.
- r(b): t(b,Y) ∉ t для Y∈v → not(t(b,b)) истинно → r(b) ✓.
- s: r(Y)∧t(Y,X) — только Y∈{b,c}, t(b,*) и t(c,*) пусты → s={} ✓.

Если убрать t(a,d), то not(t(a,d)) становится истинным и r(a) обязано сработать — получается стратифицированная модель M_1. M_1 и M_2 несравнимы по включению (M_1 содержит r(a), M_2 — нет; M_2 содержит t(a,d), M_1 — нет). Обе минимальны.

Задание 5 (*): domain-independent, но небезопасная программа

```
q(X) :- r(X,Y), not(s(Z)).
```

Небезопасна: Z встречается только в отрицательном литерале `not(s(Z))`, не связан ни одним положительным литералом.

При этом domain-independent: результат q зависит только от того, пуст ли `s` и каковы факты в `r`:

- Если s пусто: q(X) = {X | ∃Y: r(X,Y)}

- Если s непусто: q(X) = {} (для любого Z выполняется s(Z), значит ¬s(Z) ложно)

В обоих случаях ответ определяется исключительно константами, уже присутствующими в EDB — не зависит от расширения домена произвольными элементами.